

جامعة محمد الأول بوجدة
ⵜⴰⵎⴻⵔⴰⵏⵜ ⵏ ⵎⵓⵎⴻⵎⴻⵔ ⵔⵉⵎⴻⵔ ⵓⵓⵓⴷⴰ
UNIVERSITE MOHAMMED PREMIER OUJDA



المكرمة الوطنية للتكاهن الاصصاعى والرقمنة بركان
ⵜⴰⵎⴻⵔⴰⵏⵜ ⵏ ⵎⵓⵎⴻⵎⴻⵔ ⵔⵉⵎⴻⵔ ⵓⵓⵓⴷⴰ ⵏ ⵎⵓⵎⴻⵔ ⵔⵉⵎⴻⵔ ⵓⵓⵓⴷⴰ
École Nationale de l'Intelligence Artificielle et du Digital Berkane

Génie Logiciel

Pr. Houda BENHAR
Année universitaire 2023-2024

Introduction au GL – Crise du logiciel

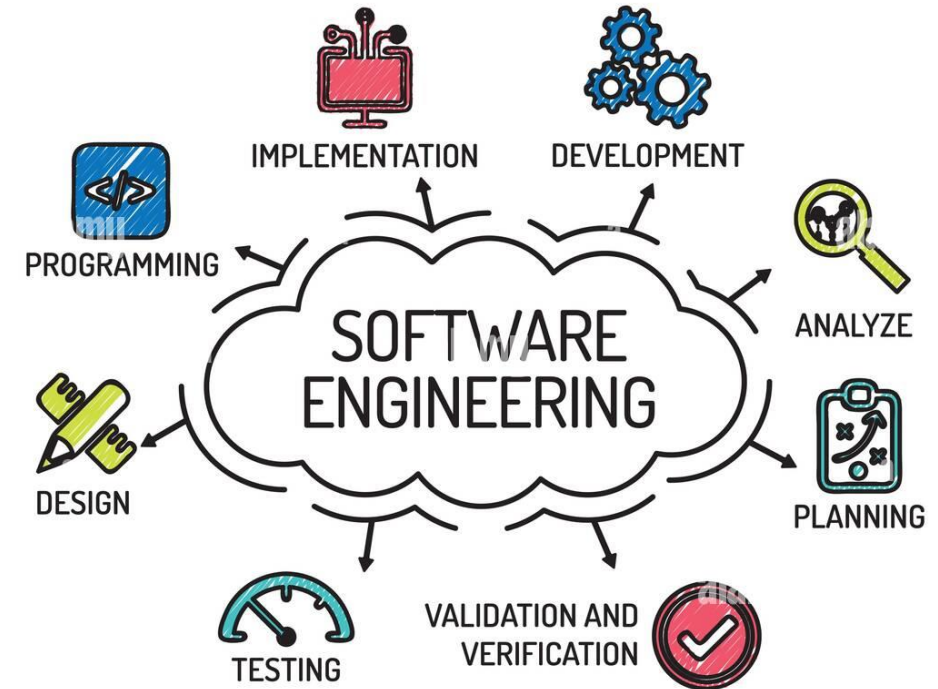
- Vers la fin des années 1960, un constat se fait jour : **le développement des grands projets informatiques est de moins en moins bien maîtrisé:**
 - ✓ Le logiciel produit ne répond pas toujours aux attentes
 - ✓ Les estimations de coût et de délai de production se révèlent inférieures à la réalité, souvent d'un facteur de 2 à 4.
 - ✓ Les équipes de programmeurs sont confrontées à des problèmes de communication
 - ✓ Les clients arrivent difficilement à décrire leurs besoins de façon assez claire pour les fournisseurs
 - ✓ Les difficultés de maintenance et d'évolution



Faute d'outils et de méthodes adaptés

Introduction au GL

- Naissance du génie logiciel (en anglais software engineering) comme discipline formelle:
 - ✓ Etablissement des principes et des pratiques pour standardiser le développement de logiciels
 - ✓ Développement de modèles de cycle de vie du logiciel, comme le modèle en cascade, pour améliorer la planification et le contrôle des projets logiciels

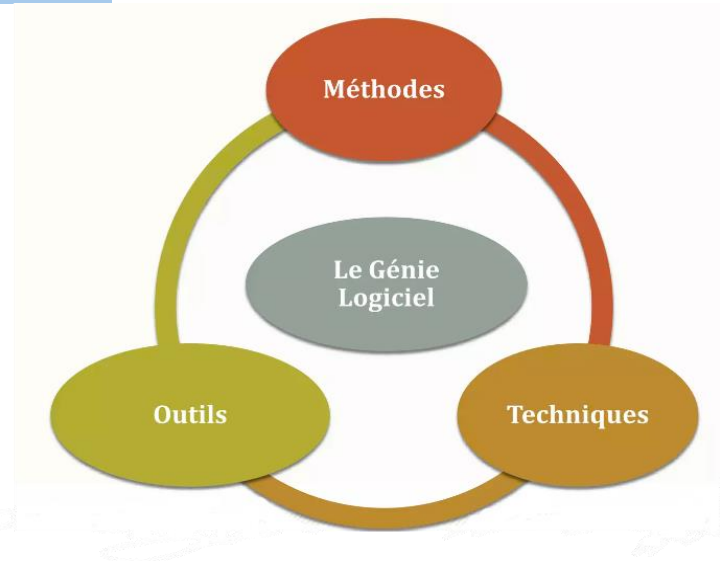


Introduction au GL

Le terme génie logiciel désigne l'ensemble des méthodes, des techniques et outils nécessaires à la construction de logiciels de haute qualité

L'objectif du génie logiciel est de permettre le développement de logiciels :

- ✓ Satisfaisant le client et le fournisseur
- ✓ De qualité supérieure
- ✓ Avec des coûts acceptables
- ✓ Dans des délais raisonnables





Qualité d'un logiciel

La qualité d'un logiciel peut être évaluée en fonction de plusieurs critères

- **Validité** : aptitude d'un produit logiciel à remplir exactement ses fonctions, définies par le cahier des charges et les spécifications.
 - **Fiabilité ou robustesse** : aptitude d'un produit logiciel à fonctionner dans des conditions anormales.
 - **Extensibilité (maintenance)** : facilité avec laquelle un logiciel se prête à sa maintenance, c'est-à-dire à une modification ou à une extension des fonctions qui lui sont demandées.
 - **Efficacité** : utilisation optimale des ressources matérielles.
 - **Intégrité** : aptitude d'un logiciel à protéger son code et ses données contre des accès non autorisés.
-

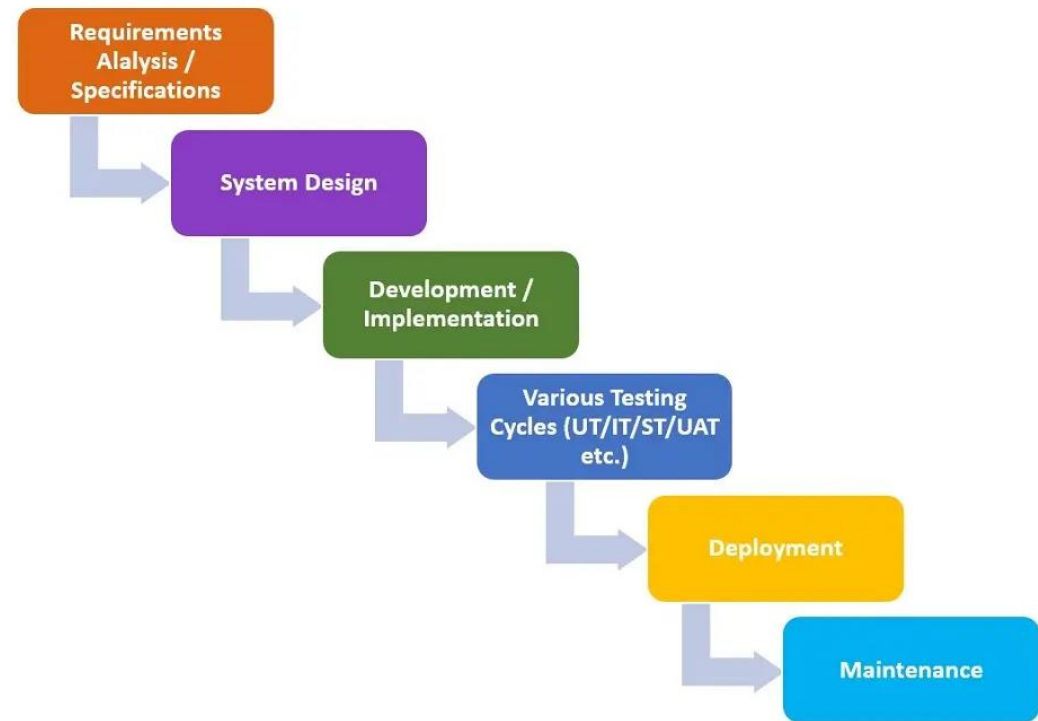


Cycle de vie d'un logiciel

- Le cycle de vie d'un logiciel (en anglais *software lifecycle*) est une représentation conceptuelle des différentes phases par lesquelles passe un logiciel depuis sa conception initiale jusqu'à sa mise hors service.
 - Il consiste à décomposer le processus de développement en modules plus petits qui peuvent être attribués, complétés et mesurés pour rendre l'ensemble plus gérable.
 - Il se concentre sur les phases suivantes du développement logiciel :
 - **Définition des besoins**
 - **Analyse**
 - **Conception**
 - **Réalisation**
 - **Tests du logiciel**
 - **Maintenance**
-

Le modèle "waterfall"

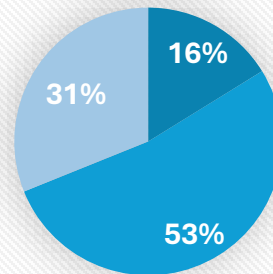
- Le modèle en cascade (waterfall en anglais) est apparu en réponse à la crise du logiciel dans les années 1970
- Processus séquentiel : un ensemble d'étapes qui coulent vers le bas (comme une chute d'eau)
- Chaque phase doit être complétée avant de passer à la suivante
- Une planification détaillée est nécessaire au début



Major-release

Etude du Standish Group (1995)

Taux de succès des projets informatiques



- Conformes aux prévisions initiales
- Dépassements en coût et délai avec diminution du nombre des fonctions offertes
- Abandonnés durant leur développement

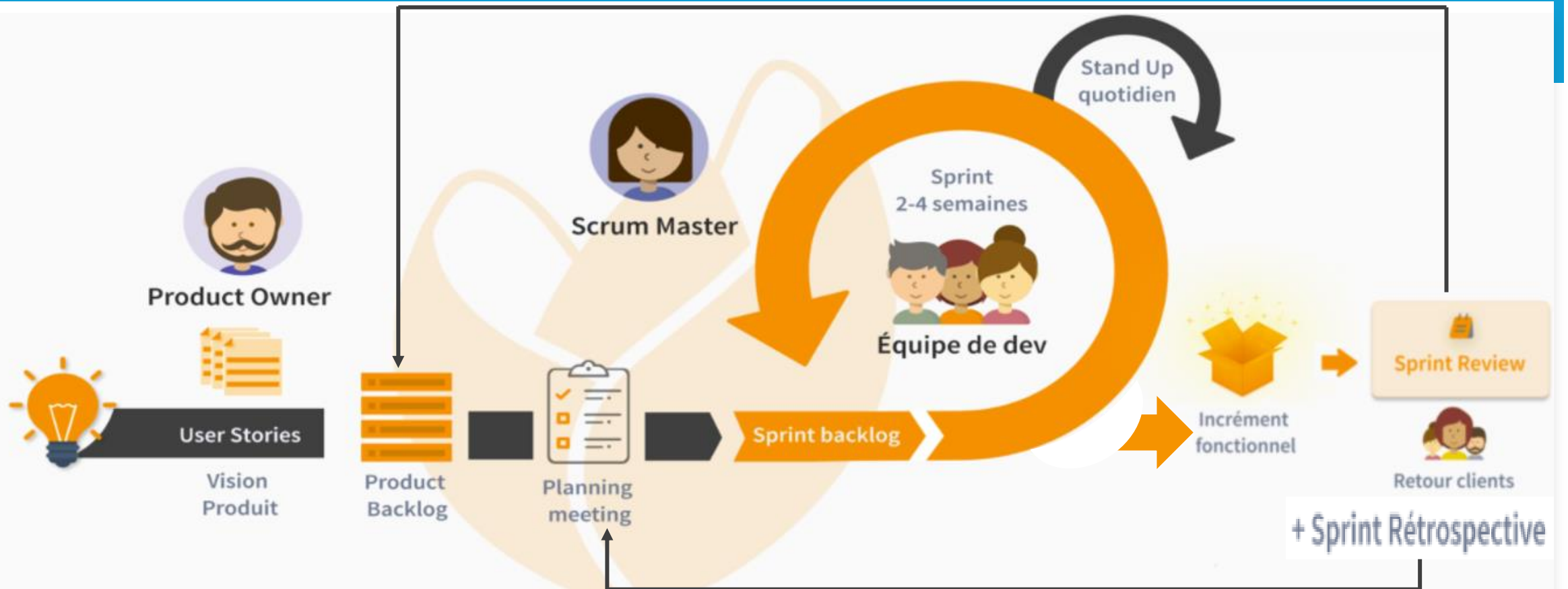
Modèle agile

- Les modèles agiles ont commencé à prendre forme, Avec le Manifeste Agile publié en 2001.
 - Le Manifeste Agile définit un ensemble de principes et de valeurs pour le développement logiciel itératif et collaboratif mettant l'accent sur:
 - ✓ **La collaboration:** Implique le client tout au long du processus de développement pour obtenir un feedback continu
 - ✓ **La flexibilité:** Capacité à s'adapter aux changements de besoins ou de priorités du client grâce à des processus itératifs et incrémentaux
 - ✓ **L'amélioration continue:** Après chaque itération, une rétrospective est menée pour identifier ce qui a bien fonctionné et ce qui peut être amélioré dans le processus de développement.
 - ✓ **La livraison rapide de produits de haute qualité (Micro release)**
-

Méthodologies agiles

- Les frameworks ou méthodologies agiles les plus populaires incluent **Scrum**, **Kanban**, **Lean Software Development**, et **Crystal**, **Extreme Programming (XP)**, entre autres.
- Ces méthodes partagent des principes fondamentaux similaires tout en offrant des approches spécifiques pour la gestion de projet et le développement logiciel.
- Scrum est largement adopté dans l'industrie du développement logiciel et bénéficie d'une vaste communauté de praticiens, de certifications, et de ressources de formation.
- Scrum offre une structure simple et facile à comprendre, avec des rôles clairement définis

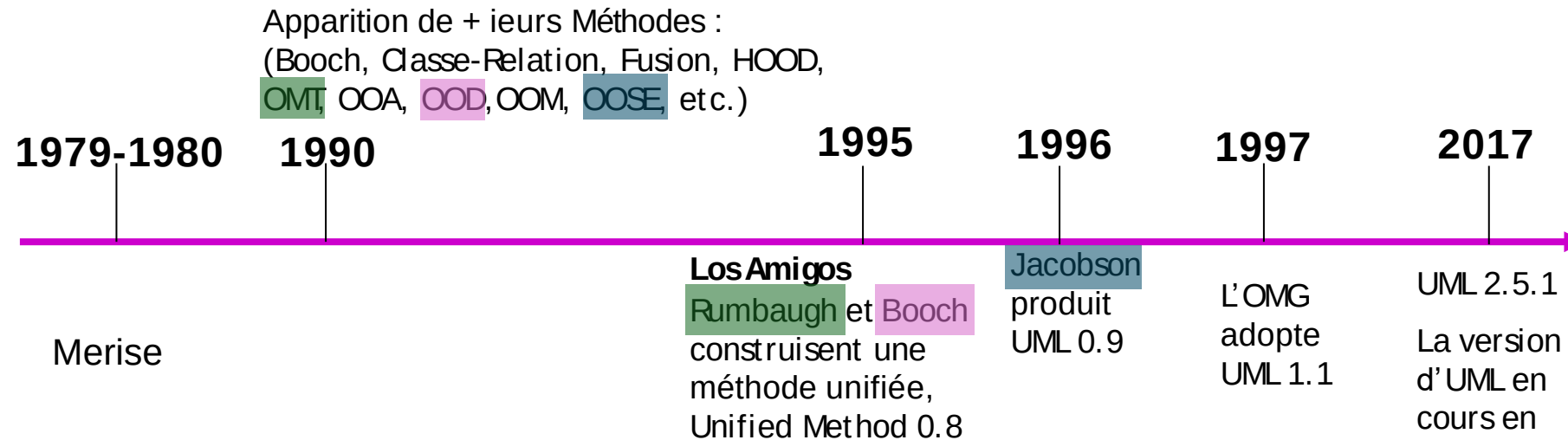
Framework scrum



UNIFIED MODELING LANGUAGE (UML)

- UML n'est pas une méthode, mais plutôt un langage graphique accompagné de textes explicatifs associés à ses diagrammes, permettant de représenter et de communiquer les divers aspects d'un système d'information.
- Il peut être intégré à plusieurs étapes pour améliorer la compréhension et la communication des aspects techniques et fonctionnels du logiciel.
 - ✓ **Préparation du Product Backlog:** visualiser les exigences et clarifier les fonctionnalités à développer (diagrammes UC)
 - ✓ **Conception de l'Architecture:** définir la structure des données et des objets (diagrammes de classes)
 - ✓ **Mise en œuvre (Sprint):** peut servir comme un outil efficace de communication entre les membres de l'équipe de développement (diagrammes de sequence)

Historique d'UML



Diagrammes UML

- Il existe un total de 14 types de diagrammes UML. Ils se répartissent en deux grands groupes:
 - ✓ Les diagrammes **comportementaux** sont focalisés sur la description de la partie **dynamique** du système à modéliser.
 - ✓ Les diagrammes UML **structurels** illustrent la **structure** d'un système, notamment les classes, les objets etc., et les relations entre ces éléments.
- Les plus utiles sont les diagrammes: d'activités, de cas d'utilisation, de classes, d'objets, de séquence et d'états-transitions.

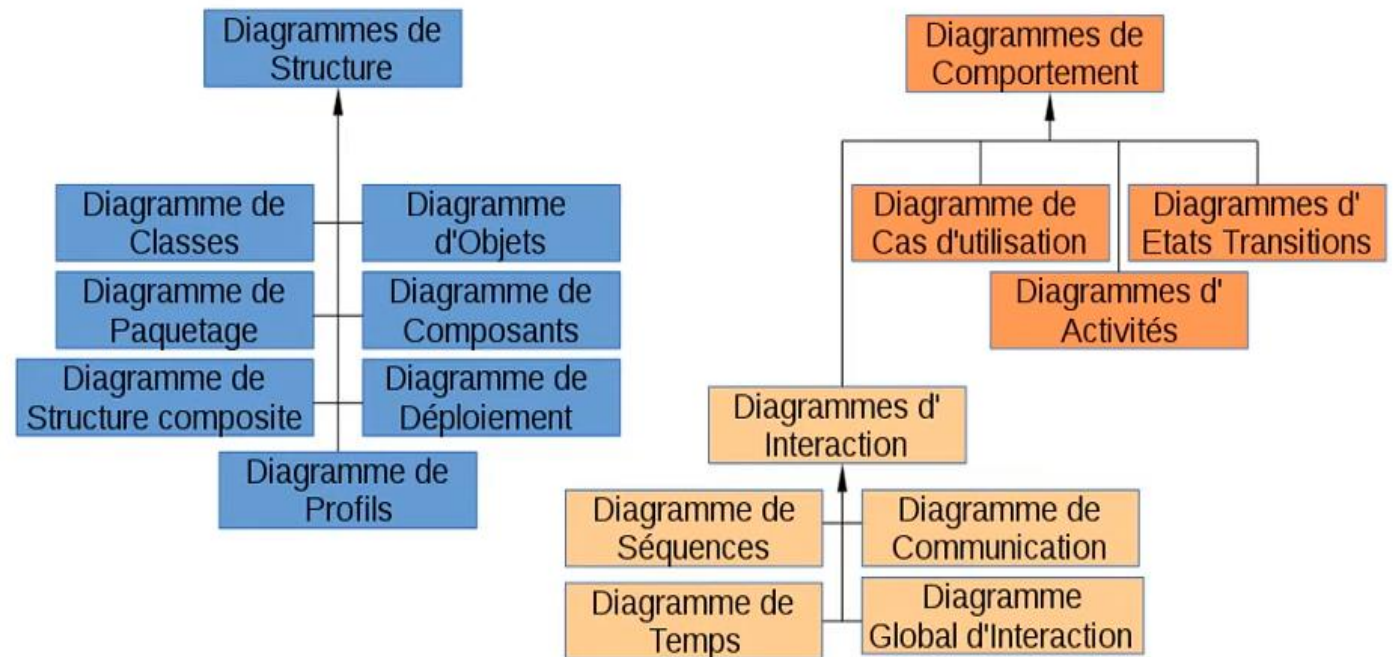


Diagramme de cas d'utilisation

- Le diagramme de cas d'utilisation (use case) est le premier diagramme à réaliser
- Les cas d'utilisation constituent un moyen de recueillir et de décrire les besoins des acteurs du système.
- Il se base sur les entretiens avec les clients, permettant ainsi de les impliquer dès les premiers stades du développement
- Il représente la structure des grandes fonctionnalités nécessaires aux utilisateurs du système
- Servir de support de référence tout au long des phases de développement du système
- La représentation d'un cas d'utilisation met en jeu 3 concepts:
 - ✓ L'acteur
 - ✓ Le cas d'utilisation
 - ✓ Les relations

Diagramme de cas d'utilisation

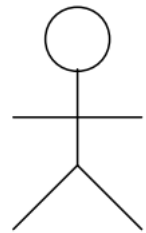
Un acteur est une entité extérieure au système modélisé, et qui interagit directement avec lui

- Un acteur correspond à un rôle, pas à une personne physique
- L'acteur est à l'origine des événements initiateurs reçus par le système
- L'acteur possède un nom : celui du rôle qu'il joue lors de son interaction avec le système
- Les acteurs peuvent être :
 - ✓ Les utilisateurs du système
 - ✓ Des logiciels, ou de es systèmes informatiques externes au système mais qui interagissent avec lui
 - ✓ Un équipement

Diagramme de cas d'utilisation

Un acteur est une entité extérieure au système modélisé, et qui interagit directement avec lui

- Il existe deux types d'acteur:
 - ✓ **Un acteur principal (Primary)** utilise le système: celui qui initie une interaction avec le système pour atteindre un but ou accomplir une tâche. C'est l'utilisateur ou le système externe qui déclenche le cas d'utilisation.
 - ✓ **Un acteur secondaire (Secondary)** participe pour la réalisation d'un cas d'utilisation: soutient ou aide à compléter le cas d'utilisation. Bien qu'il participe au cas d'utilisation, il ne l'initie pas directement.
- Un acteur est représenté par un petit bonhomme avec son nom inscrit dessous.



Acteur 1

Diagramme de cas d'utilisation

Un cas d'utilisation est une fonctionnalité ou un service fourni à un acteur par le système

- Un cas d'utilisation est représenté par un ovale
- Le nom du cas d'utilisation apparaît à l'intérieur de l'ovale

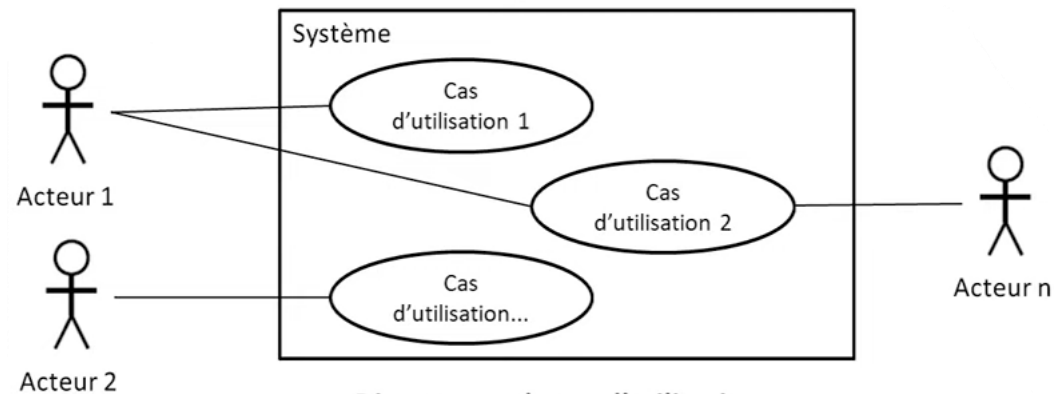
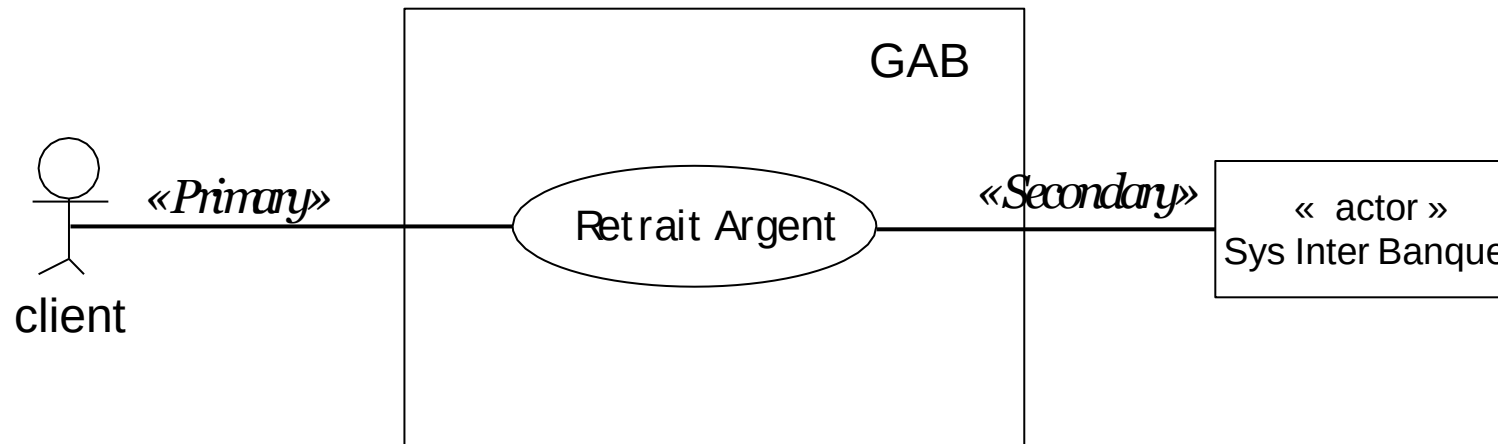


Diagramme de cas d'utilisation

Le rectangle représente les frontières du système. tout ce qui est à l'intérieur c'est le système qu'on doit développer et tout ce qui est à l'extérieur c'est l'environnement de notre système

Diagramme de cas d'utilisation

Exemple:



Il est également possible de représenter un acteur sous la forme d'un classeur stéréotypé <<actor>>

Diagramme de cas d'utilisation

Relations entre acteurs

- La seule relation possible entre deux acteurs est la généralisation
- Un acteur A est une généralisation d'un acteur B lorsque tous les cas d'utilisation accessibles à A le sont aussi à B, mais l'inverse n'est pas vrai.
- Le symbole utilisé pour la généralisation entre acteurs est une flèche avec un trait plein dont la pointe est un triangle vide désignant l'acteur le plus général

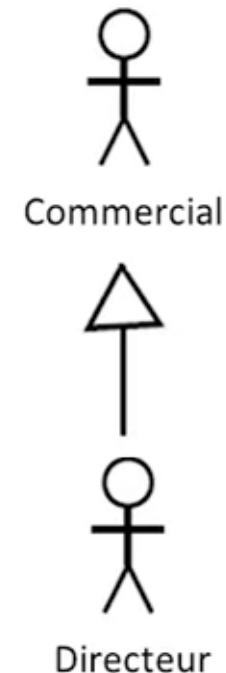


Diagramme de cas d'utilisation

- Il existe principalement deux types de **relations entre cas d'utilisation**:
 - ✓ La généralisation/spécialisation.
 - ✓ Les dépendances stéréotypées, qui sont explicitées par un stéréotype (les plus utilisés sont l'inclusion et l'extension)
- Une dépendance se représente par une flèche avec un trait pointillé



- Le symbole utilisé pour la généralisation est une flèche avec un trait plein dont la pointe est un triangle fermé désignant le cas le plus général

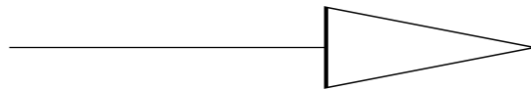


Diagramme de cas d'utilisation

Relation d'inclusion <<include>>:

- Une relation <<include>> est utilisée pour indiquer qu'un cas d'utilisation utilise **systematiquement et intégralement** une séquence d'activités décrite dans un autre cas d'utilisation
- Est représentée par une flèche pointillée étiquetée « include », pointant vers le cas d'utilisation inclus.

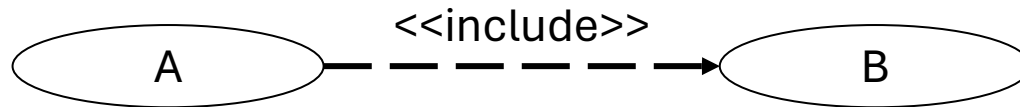


Diagramme de cas d'utilisation

Relation d'inclusion <<include>>: Exemple

- Les inclusions permettent essentiellement de factoriser une partie de la description d'un cas d'utilisation qui serait commune à d'autres cas d'utilisation

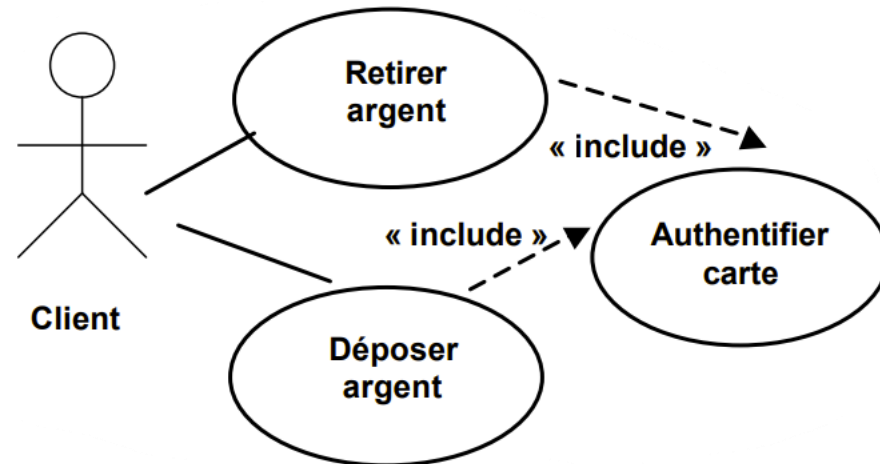
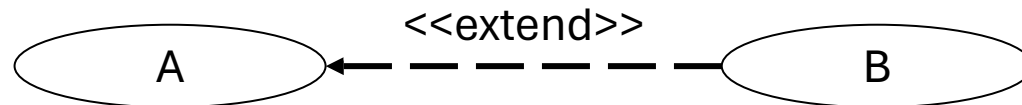


Diagramme de cas d'utilisation

Relation d'extension <<extend>>:

- Une relation <<extend>> est utilisée pour indiquer qu'un cas d'utilisation peut être appelé au cours de l'exécution d'un autre cas d'utilisation.
- Est représentée par une flèche pointillée étiquetée « extend », pointant vers le cas d'utilisation étendu



- Exécuter A peut éventuellement entraîner l'exécution de B : contrairement à l'inclusion, l'extension est optionnelle.

Diagramme de cas d'utilisation

Relation d'extension : Exemple

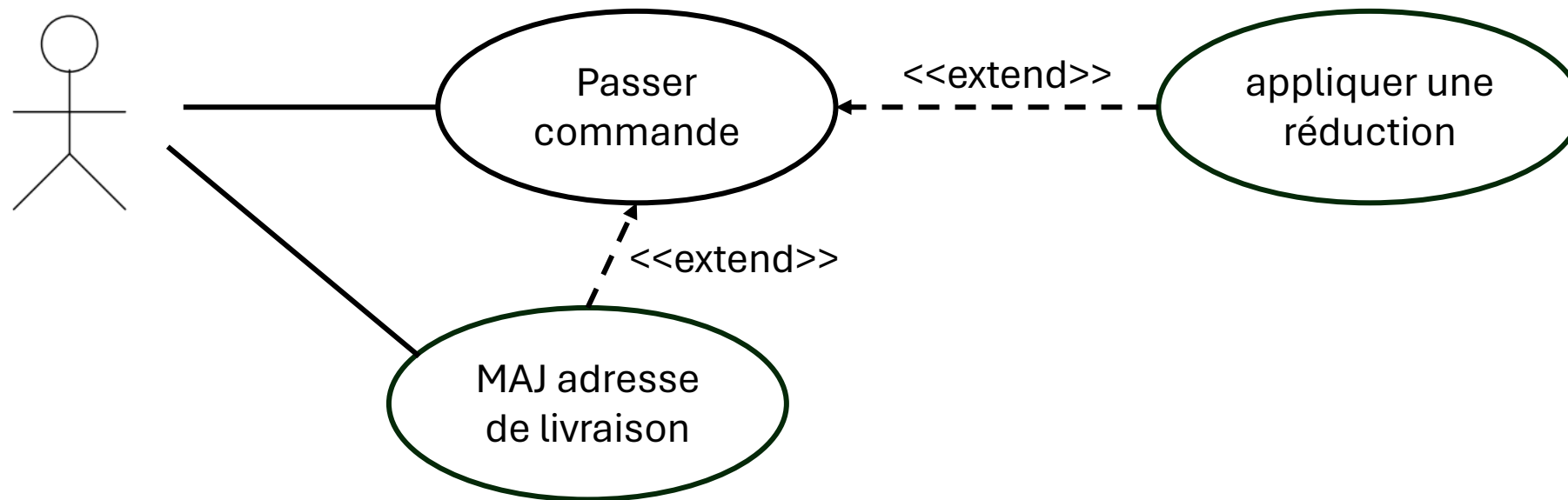


Diagramme de cas d'utilisation

Relation de généralisation:

- Une relation de généralisation indique qu'un cas d'utilisation est une spécialisation d'un autre cas d'utilisation
- Est représentée par une flèche « d'héritage » terminée par un triangle vide pointant du cas d'utilisation spécialisé vers le cas d'utilisation le plus général

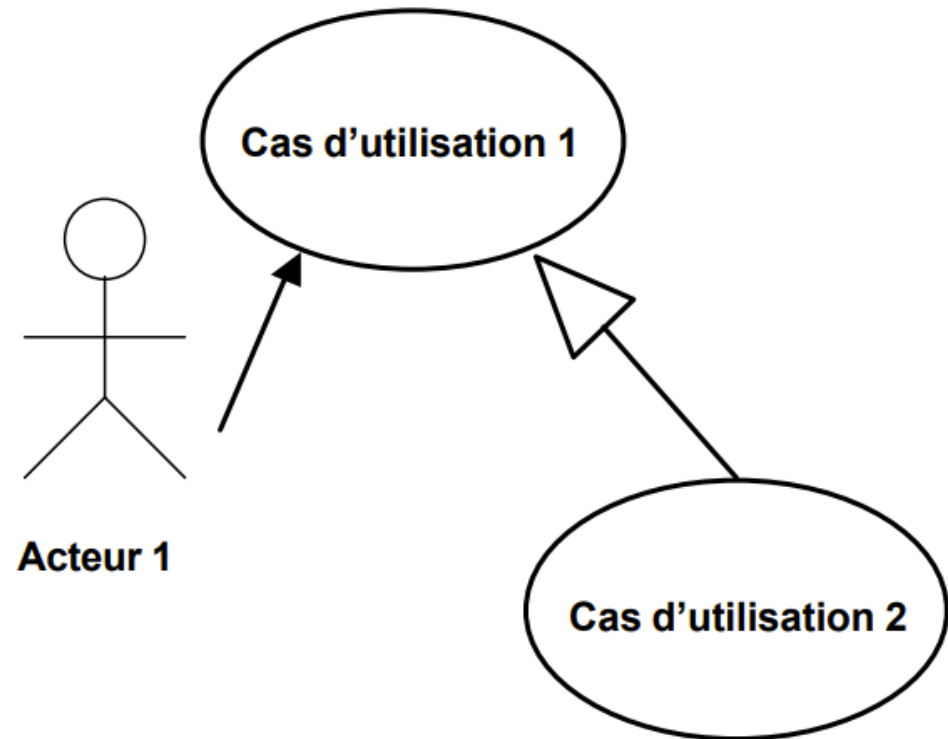
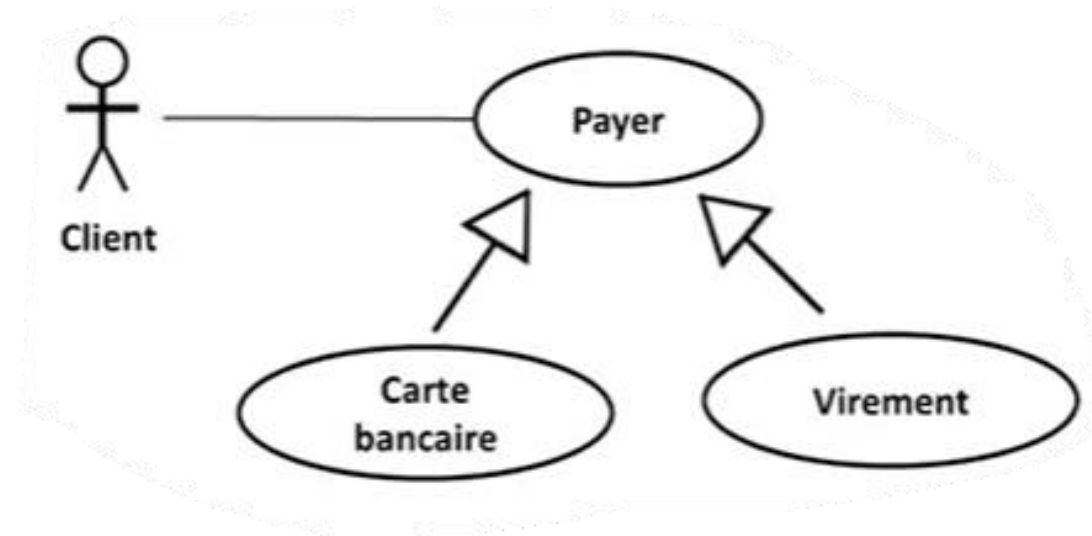


Diagramme de cas d'utilisation

Relation de généralisation:

- Le paiement par carte bancaire est un cas particulier du paiement



Description textuelle des cas d'utilisation

- Le diagramme de cas d'utilisation décrit les grandes fonctions d'un système du point de vue des acteurs, mais n'expose pas de façon détaillée le dialogue entre les acteurs et les cas d'utilisation
 - Un UC se présente généralement sous forme textuelle, même s'il est parfois représenté sous forme de diagramme de séquences ou d'activités UML
- **Titre** (commence par un verbe)
 - **Objectif** (descriptif court : une phrase si possible)
 - **Acteurs**
 - **Dates**
 - **Responsable**
 - **Version**
 - **Pré-conditions** – conditions nécessaires pour que le cas d'utilisation s'exécute
 - **Scénarios**: nominal, alternatif, d'erreur
 - **Exceptions**
 - **Post-conditions** – état d'une partie du système après l'exécution du cas d'utilisation
-

Description textuelle des cas d'utilisation

Exemple d'un cas d'utilisation décrit sous une forme textuelle

Titre : Retirer de l'argent avec une carte Visa

Objectif : ce cas d'utilisation permet à un porteur de carte Visa, non client de la banque, de retirer de l'argent, si son crédit hebdomadaire le permet.

Acteurs : Porteur de CB Visa (principal), SA Visa(secondaire).

Date de création : 02/03/00

Date de mise à jour : 09/11/00

Version : 2.2

Responsable : Pascal Roques

Préconditions:

- La caisse du GAB est alimentée,
- Aucune carte bancaire n'est dans le lecteur

Description textuelle des cas d'utilisation

Exemple d'un cas d'utilisation décrit sous une forme textuelle

Scénario nominal :

1. Le porteur de CB Visa introduit sa carte Visa dans le lecteur de cartes du GAB.
2. Le GAB vérifie que la carte introduite est bien une carte Visa.
3. Le GAB demande au porteur de CB Visa de saisir son code d'identification.
4. Le porteur de CB Visa saisit son code d'identification.
5. Le GAB compare le code d'identification avec celui codé sur la puce de la carte.
6. Le GAB demande une autorisation au système d'autorisation VISA.
7. Le système d'autorisation VISA donne son accord et indique le solde hebdomadaire.
8. Le GAB demande au porteur de CB Visa de saisir le montant désiré du retrait.
9. Le porteur de CB Visa saisit le montant désiré du retrait.
10. Le GAB contrôle le montant demandé par rapport au solde hebdomadaire.
9. Le GAB demande au porteur de CB Visa s'il veut un ticket.
11. Le porteur de CB Visa demande un ticket.
12. Le GAB rend sa carte au porteur de CB Visa
13. Le porteur de CB Visa reprend sa carte.
14. Le GAB délivre les billets et un ticket.
15. Le porteur de CB Visa prend les billets et le ticket.

Description textuelle des cas d'utilisation

Exemple d'un cas d'utilisation décrit sous une forme textuelle

Enchaînements alternatifs:

A1 : code d'identification provisoirement erroné

L'enchaînement A1 démarre au point 5 du scénario nominal.

6. Le GAB indique au client que le code est erroné, pour la première ou deuxième fois.

7. Le GAB enregistre l'échec sur la carte.

Le scénario nominal reprend au point 3.

A2 : montant demandé supérieur au solde hebdomadaire

L'enchaînement A2 démarre au point 10 du scénario nominal.

11. Le GAB indique au client que le montant demandé est supérieur au solde hebdomadaire.

Le scénario nominal reprend au point 3.

Description textuelle des cas d'utilisation

Exemple d'un cas d'utilisation décrit sous une forme textuelle

Enchaînements d'exception:

E1 : carte invalide

L'enchaînement E1 démarre au point 2 du scénario nominal.

3. Le GAB indique au porteur que la carte est invalide (illisible, périmée, etc.), la confisque et le cas d'utilisation est terminé.

E2 : code d'identification définitivement erroné

L'enchaînement E2 démarre au point 5 du scénario nominal.

6. Le GAB indique au client que le code est erroné, pour la troisième fois.

7. Le GAB confisque la carte.

8. Le système d'autorisation VISA est informé et le cas d'utilisation est terminé

Postconditions :

La caisse du GAB contient moins de billets qu'au début du cas d'utilisation

Outils de modélisation



ArgoUML : Open source et gratuit



Disponible à la fois en
versions payantes et gratuites



Très complet pour les professionnels



StarUML: propose une
version d'essai gratuite

Exercice

Enoncé: On souhaite mettre en place un système de gestion de bibliothèque afin d'automatiser le suivi des adhérents et des livres empruntés. Le système à concevoir doit assister le bibliothécaire, chargé de gérer les inscriptions, les emprunts (validation, annulation, modification), les réservations (listing, approbation, annulation), ainsi que l'enregistrement des restitutions et la mise à jour de la liste des exemplaires.

Un visiteur peut consulter un ouvrage. Un emprunteur doit s'inscrire et payer les frais d'inscription pour devenir adhérent avant d'emprunter un ouvrage.

Un emprunteur peut réserver un livre qui est indisponible (déjà prêté ou répertorié mais non encore acheté). Cette réservation peut être annulée à tout moment (par l'emprunteur lui-même ou par le bibliothécaire). Un emprunteur peut également prolonger la durée d'un emprunt si aucune réservation n'a été faite sur le livre. Il doit retourner les ouvrages empruntés avant leur date limite de restitution.

Un gestionnaire est chargé de relancer les lecteurs qui n'ont pas rendu leurs ouvrages dans le délai autorisé. Il a également la possibilité de passer une sanction à un adhérent en cas de non-respect du délai de retour.

1. Modéliser cette situation par un diagramme de cas d'utilisation